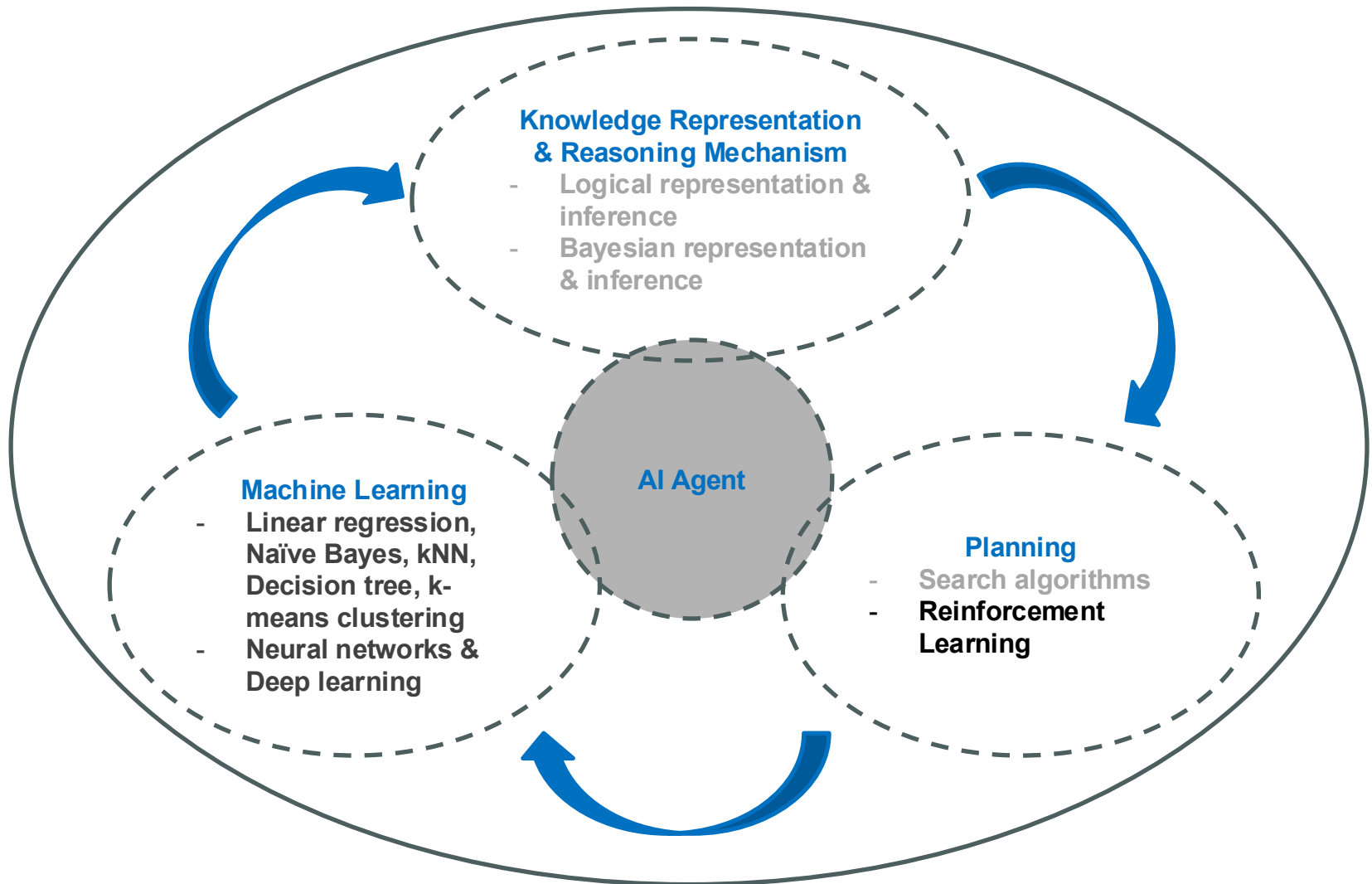


FIT3080 ARTIFICIAL INTELLIGENCE
Week 09, Semester 2, 2026
Reinforcement Learning

Jianfei Cai
Faculty of Information Technology
Monash University



Overview of Unit topics

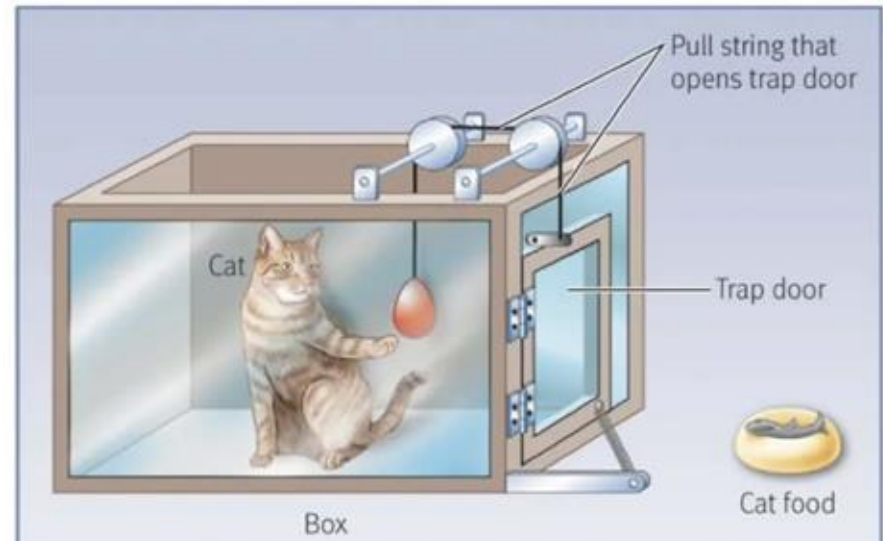


Brief History & Success

- Analogies with animal learning and psychology
- Minsky's PhD thesis (1954): Stochastic Neural-Analog Reinforcement Computer
- Temporal difference (TD) learning - Gammon (Tesauro, 1992) – big success story
- Job-shop scheduling for NASA space missions (Zhang and Dietterich, 1997)
- Robotic soccer (Stone and Veloso, 1998) – part of the world-champion approach
- AlphaGo – Google Deepmind, 2016

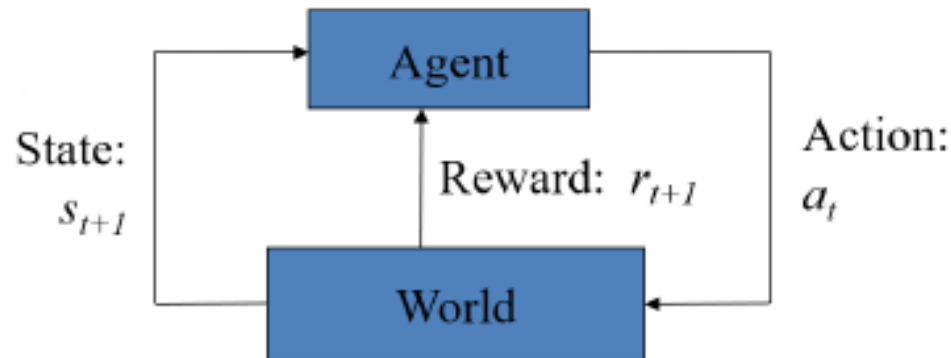


Image from physiotherapy-burlington.ca



Thorndike's Law of Effect (Image by [Dyar Jabbary](#) on [ResearchGate](#))

Reinforcement Learning (RL) framework

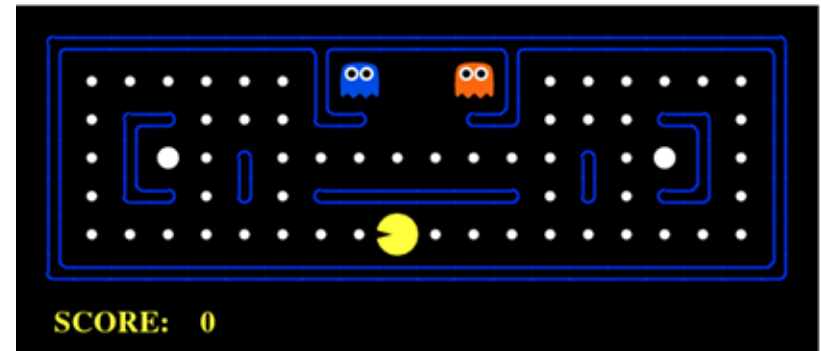


Agent and environment interact at discrete time steps: $t = 0, 1, 2, \dots$

- Agent observes state at step t : $s_t \in S$
- produces action at step t : $a_t \in A(s_t)$
- gets resulting reward: $r_{t+1} \in \mathbb{R}$
- and resulting next state: s_{t+1}

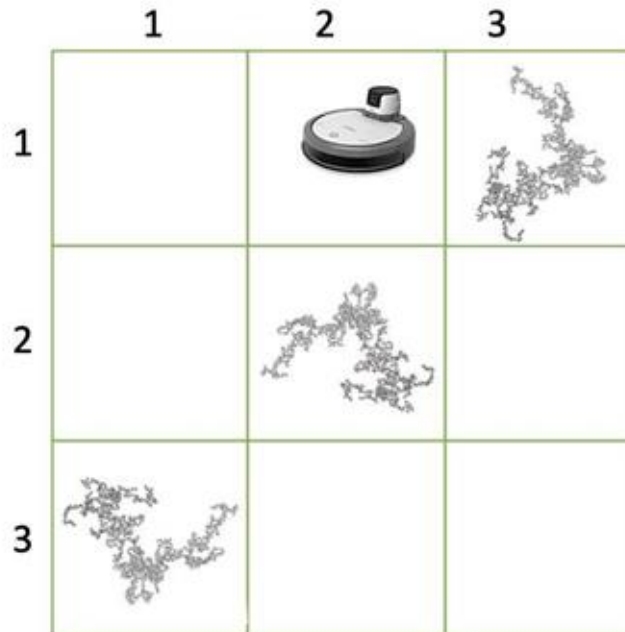
Elements of Reinforcement Learning (RL)

- **4 main elements:**
 - **Policy:**
 - learning agent's way of behaving at a given time
 - **Reward:**
 - Goal of a reinforcement learning problem
 - primary way for impacting the policy
 - **Value function:**
 - what is good in the long run.
 - **Model:**
 - Dynamics of the environment.
- Example: PacMan



- Dilemma of exploring new states while maximizing its reward at the same time
- **Exploration vs Exploitation trade-off.**

Example: Vacuum cleaner



- **9 locations:** squares labelled by coordinates (i, j) where $i = 1, 2, 3$. and $j = 1, 2, 3$.
- **Actions:** move left, right, up or down, vacuum the dirt, or do nothing
- **Strategy:**
 - give a **small positive** reward whenever it cleans *a square*
 - **small negative reward** if the agent attempts to clean an already cleaned square
 - **big positive reward** when all squares are cleaned

Two Types of Reinforcement

Positive

When an event, occurs due to a particular behaviour, increases the strength and the frequency of the behaviour. it has a positive effect on behaviour.

Advantages:

- Encourages exploration, leading to the discovery of optimal strategies.
- Reinforces desirable behaviours, promoting long-term improvement.

Disadvantages:

- Doesn't penalize bad actions, slowing the learning of what to avoid.
- Can lead to risky exploration if negative consequences are not considered.

Negative

Strengthening of a behaviour because a negative condition is stopped or avoided.

Advantages:

- Quickly teaches the agent to avoid harmful actions.
- Focuses on risk management, promoting safer strategies.

Disadvantages:

- Discourages exploration, leading to overly cautious behaviour.
- Lacks reinforcement for positive actions, slowing progress toward optimal solutions.

Examples

- Fly stunt manoeuvres in a helicopter
 - +ve reward for following desired trajectory
 - -ve reward for crashing
- Defeat the world champion at Go
 - +ve/-ve reward for winning/losing a game
- Make a humanoid robot walk
 - +ve reward for forward motion
 - -ve reward for falling over
- Play Atari games better than humans
 - +ve/-ve reward for increasing/decreasing score

Exploitation vs Exploration

Online decision-making involves a fundamental choice:

- **Exploitation:** Make the best decision given current information
- **Exploration:** Gather more information

The best long-term strategy may involve short-term sacrifices

Gather enough information to make the best overall decisions

Example 1: Restaurant Selection

- **Exploitation-** Go to your favourite restaurant
- **Exploration-** Try a new restaurant

Example 2: Online Banner Advertisements

- **Exploitation-** Show the most successful advert
- **Exploration-** Show a different advert

Example 3: Oil Drilling

- **Exploitation-** Drill at the best known location
- **Exploration-** Drill at a new location

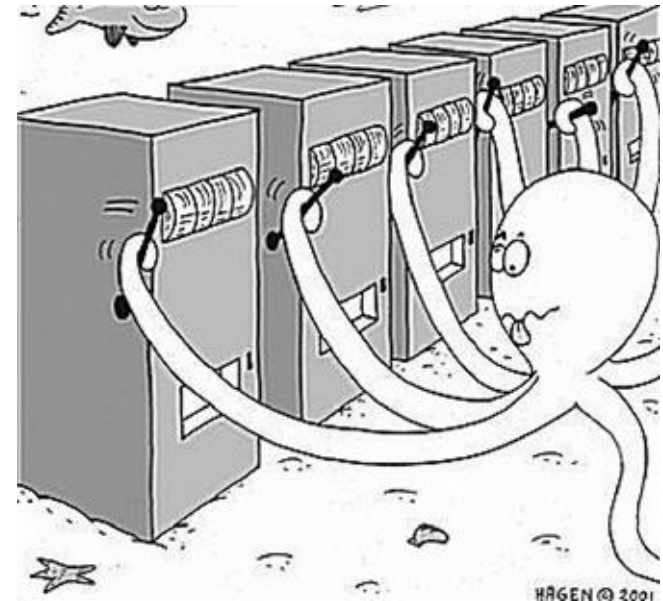
Example 4: Game Playing

- **Exploitation-** Play the move you believe is best
- **Exploration-** Play an experimental move

Multi-armed Bandits

Multi-armed Bandits

- ❖ Models an agent
 - ❖ Attempt to acquire new knowledge (called "exploration")
 - ❖ And optimize their decisions based on existing knowledge (called "exploitation")
- ❖ Origin :
 - ❖ A gambler at a row of slot machines (sometimes known as "one-armed bandits"), has to:
 - ❖ Decide which machines to play?
 - ❖ How many times to play each machine and in which order to play them?
 - ❖ Whether to continue with the current machine or try a different machine?



The Multi-Armed Bandit

- In k -armed bandit problem,
 - *Each of k actions* has an expected or mean reward
 - This is the *value* of that action
 - A_t = selected action on time step t
 - R_t = corresponding reward
 - $q_*(a)$ the expected reward given that a is selected.
 - $q_*(a) \doteq \mathbb{E}[R_t | A_t = a]$
 - always select the action with highest value
 - $Q_t(a)$ denotes the estimated value of action a at time step t .
 - Preferable: $Q_t(a)$ close to $q_*(a)$

The Multi-Armed Bandit

- **Greedy actions**
 - Action whose estimated value is the greatest
 - If selected,
 - Exploiting current knowledge of the values of the actions
 - If select one of the nongreedy actions,
 - Exploring
- To explore or exploit
 - Depends in a complex way on the precise values of the estimates, uncertainties, and the number of remaining steps.

The Multi-Armed Bandit

- **Action- value methods**

- true value of an action is the mean reward when that action is selected

- $Q_t(a) \doteq \frac{\text{sum of rewards when } a \text{ taken prior to } t}{\text{no. of times } a \text{ taken prior to } t} = \frac{\sum_{i=1}^{t-1} R_i \cdot \mathbb{I}_{A_t=a}}{\sum_{i=1}^{t-1} \mathbb{I}_{A_t=a}},$

- $\mathbb{I}_{\text{predicate}}$: random variable

- =1, If predicate is true
- =0, If predicate is not true

- If denominator:

- =0, $Q_t(a)$ as some default value (e.g. 0)
- $=\infty$, $Q_t(a)$ converges to $q_*(a)$

- This is sample-average method

- for estimating action values because each estimate is an average of the sample of relevant rewards

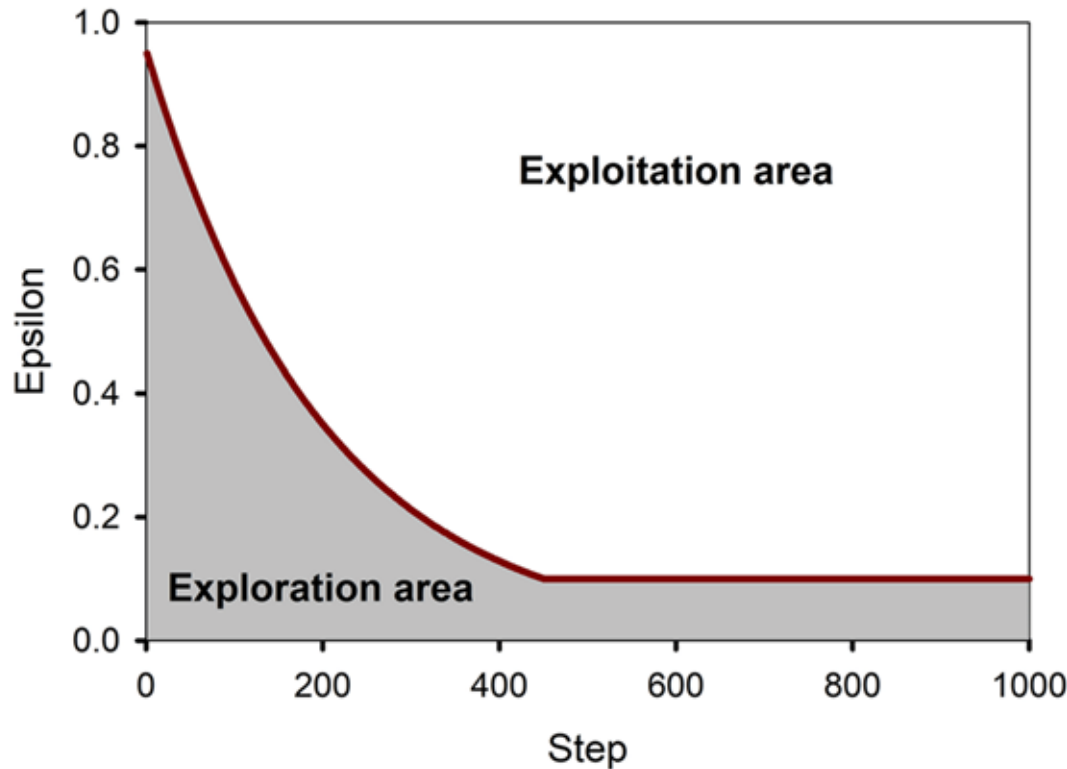
ϵ -greedy (epsilon greedy) Selection

- ϵ –greedy (epsilon greedy)
 - behave greedily but
 - with small probability ϵ , it select randomly from among all the actions with equal probability, independently of the action-value estimates.
 - This algorithm continues to explore forever
 - With probability $1 - \epsilon$, select $A_t \doteq \underset{a}{\operatorname{argmax}} Q_t(a)$
 - With probability ϵ select a random action

Action at time(t)	{	$\max Q_t(a)$	with probability $1-\epsilon$
		any action (a)	with probability ϵ

ϵ -greedy (epsilon greedy) Selection

Epsilon greedy method



- At each step, a random number is generated by the model.
- If the number was lower than epsilon in that step (exploration area)
 - the model chooses a random action
- if it was higher than epsilon in that step (exploitation area)
 - the model chooses an action based on what it learned.

Exercise 1:

In ϵ -greedy action selection, for the case of two actions and $\epsilon = 0.5$ (or 50%), what is the probability that the greedy action is selected?

The Multi-Armed Bandit

- Given Q_n and the n th reward, R_n , the new average of all n rewards can be computed by

$$Q_n \doteq \frac{R_1 + R_2 + \cdots + R_{n-1}}{n-1}$$

$$\begin{aligned} Q_{n+1} &= \frac{1}{n} \sum_{i=1}^n R_i \\ &= \frac{1}{n} \left(R_n + \sum_{i=1}^{n-1} R_i \right) \\ &= \frac{1}{n} \left(R_n + (n-1) \frac{1}{n-1} \sum_{i=1}^{n-1} R_i \right) \\ &= \frac{1}{n} (R_n + (n-1)Q_n) \\ &= \frac{1}{n} (R_n + nQ_n - Q_n) \\ &= Q_n + \frac{1}{n} [R_n - Q_n], \end{aligned}$$

which holds even for $n = 1$, obtaining $Q_2 = R_1$ for arbitrary Q_1

The Multi-Armed Bandit

Example: Consider the Multi-Armed Bandit algorithm. $Q_t(a)$ at $t = 20$ for 4 actions $[a_1, a_2, a_3, a_4]$ is $[0.75, 0.70, 0.45, 0.55]$ and the number of times which each action is selected, is equal to $[6, 6, 5, 3]$. Assume for the next iteration, ϵ -greedy action selection select the following actions (a_1) with the gain rewards = 2.

$$Q_n \doteq \frac{R_1 + R_2 + \cdots + R_{n-1}}{n - 1}$$

$$\begin{aligned} Q_{n+1} &= \frac{1}{n} \sum_{i=1}^n R_i \\ &= \frac{1}{n} \left(R_n + \sum_{i=1}^{n-1} R_i \right) \\ &= \frac{1}{n} \left(R_n + (n-1) \frac{1}{n-1} \sum_{i=1}^{n-1} R_i \right) \\ &= \frac{1}{n} (R_n + (n-1)Q_n) \\ &= \frac{1}{n} (R_n + nQ_n - Q_n) \\ &= Q_n + \frac{1}{n} [R_n - Q_n], \end{aligned}$$

The Multi-Armed Bandit

A simple bandit algorithm

Initialize, for $a = 1$ to k :

$$Q(a) \leftarrow 0$$

$$N(a) \leftarrow 0$$

Repeat forever:

$$A \leftarrow \begin{cases} \arg \max_a Q(a) & \text{with probability } 1 - \varepsilon \quad (\text{breaking ties randomly}) \\ \text{a random action} & \text{with probability } \varepsilon \end{cases}$$

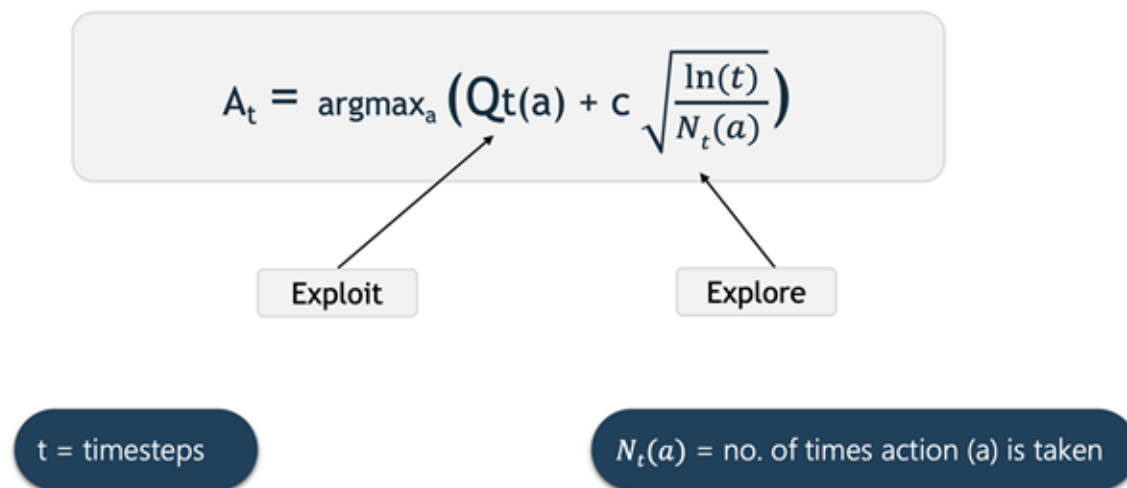
$$R \leftarrow \text{bandit}(A)$$

$$N(A) \leftarrow N(A) + 1$$

$$Q(A) \leftarrow Q(A) + \frac{1}{N(A)} [R - Q(A)]$$

Upper Confidence Bound Action Selection

- Use uncertainty in the action-value estimates for balancing exploration and exploitation.
- Use a sampled set of rewards, thus UCB uses uncertainty in the estimates to drive exploration.

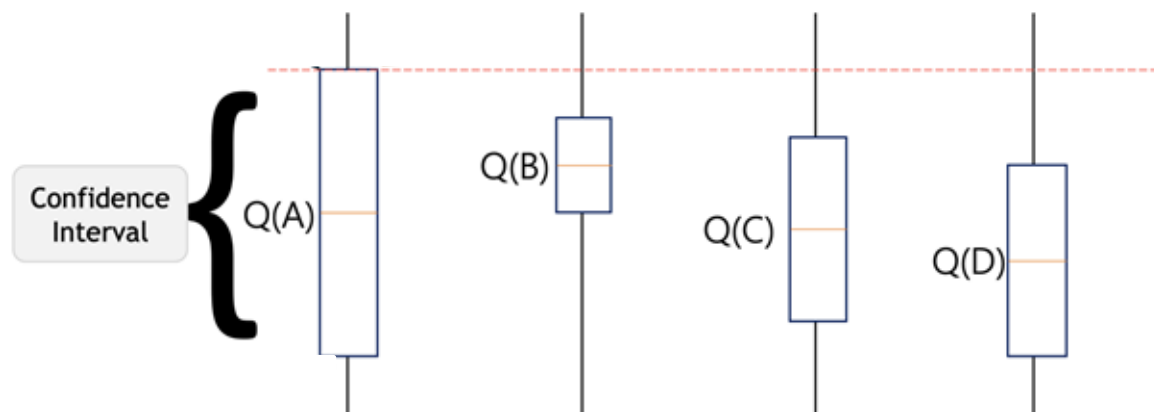


$Q_t(a)$ here represents the current estimate for action a at time t .

We select the action that has the highest estimated action-value plus the upper-confidence bound exploration term.

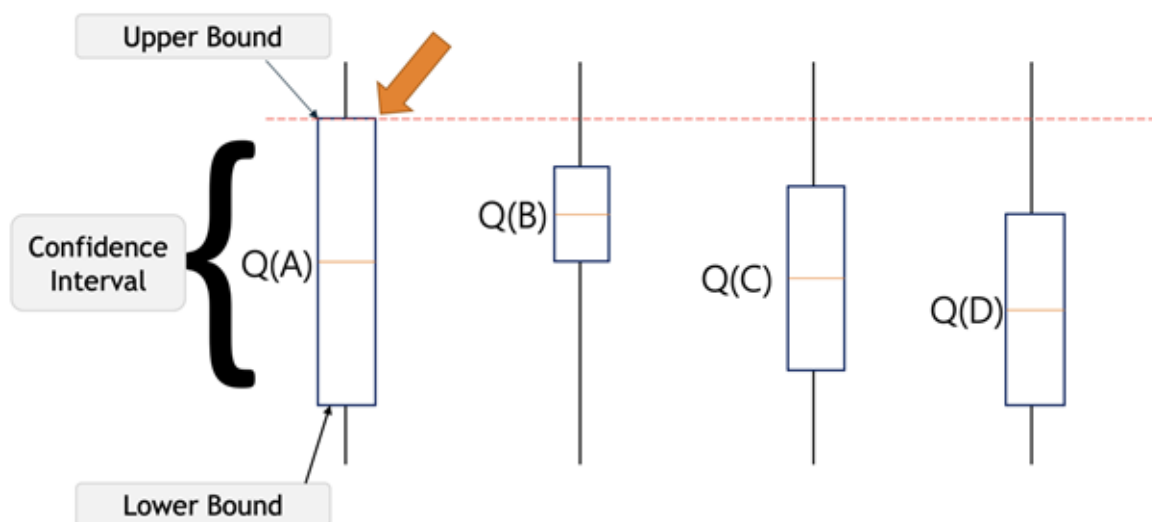
Upper Confidence Bound Action Selection

- **Upper Confidence Bound** follows the principle of optimism in the face of uncertainty which implies that if we are uncertain about an action, we should optimistically assume that it is the correct action.
- **Example :**
- There are 4 actions with associated uncertainties in the picture below, our agent has no idea which is the best action.
- Through UCB algorithm,



Upper Confidence Bound Action Selection

- **Upper Confidence Bound** follows the principle of optimism in the face of uncertainty which implies that if we are uncertain about an action, we should optimistically assume that it is the correct action.
- **Example :**
- There are 4 actions with associated uncertainties in the picture below, our agent has no idea which is the best action.
- Through UCB algorithm,
 - it will optimistically pick the action that has the highest upper bound
e.g. A



Temporal-Difference Learning

Temporal-Difference (TD) Learning

- Value functions
 - state-action pair functions that estimate how good a particular action will be in a given state, or what the return for that action is expected to be
 - $V^\pi(s)$: the value of a state s under policy π
 - The expected return when starting in s and following π thereafter
 - $Q^\pi(s, a)$: the value of taking an action a in state s under a policy π
 - The expected return when starting from s taking the action a and thereafter following policy π .
- Being used to accurately choose an action that will provide the best total reward possible, after being in that given state.

Temporal Difference (TD) Learning

- With Temporal Difference learning,
 - an estimate of final reward is calculated at each state
 - State-action value updated for every step of the way
 - $V(s_t) \leftarrow V(s_t) + \alpha[r_{t+1} + \gamma V(s_{t+1}) - V(s_t)]$
 - Where r_{t+1} is the observed reward at time $t+1$.
- TD also called as a "bootstrapping" method,
 - the value is updated partly using an existing estimate and not a final reward.

On-Policy and Off-Policy

- **On-policy**
 - learn the value of the policy that is used to make decisions
 - not so strict that it always chooses the action that gives the most reward
 - Example: SARSA
- **Off-policy**
 - can learn different policies for behaviour and estimation.
 - can update the estimated value functions using hypothetical actions
 - those which have not actually been tried
 - Example: Q-learning

State–action–reward–state–action (SARSA)

- an algorithm for learning a Markov decision process(MDP) policy
 - interacts with the environment -> updates the policy based on actions taken
- known as an on-policy learning algorithm
- Formula:

$$Q(s_t, a_t) = Q(s_t, a_t) + \alpha(r_{t+1} + \gamma Q(s_{t+1}, a_{t+1}) - Q(s_t, a_t))$$

- Where:
 - Q value for a state-action is updated by an error, adjusted by the learning rate alpha (α)
 - γ is the discount factor
- Learning rate
 - factor of 0 -> the agent not learn anything
 - factor of 1 -> the agent consider only the most recent information.
- Discount rate
 - factor of 0 ->the agent "opportunistic"
 - factor of 1->it strive for a long-term high reward
 - If meets/exceeds 1, the Q values may diverge

State–action–reward–state–action (SARSA)

Sarsa (on-policy TD control) for estimating $Q \approx q_*$

Initialize $Q(s, a), \forall s \in \mathcal{S}, a \in \mathcal{A}(s)$, arbitrarily, and $Q(\text{terminal-state}, \cdot) = 0$

Repeat (for each episode):

 Initialize S

 Choose A from S using policy derived from Q (e.g., ϵ -greedy)

 Repeat (for each step of episode):

 Take action A , observe R, S'

 Choose A' from S' using policy derived from Q (e.g., ϵ -greedy)

$Q(S, A) \leftarrow Q(S, A) + \alpha [R + \gamma Q(S', A') - Q(S, A)]$

$S \leftarrow S'; A \leftarrow A';$

 until S is terminal

- always choose the action $A(t+1)$ of state $S(t+1)$ first in state S_t under the policy.
- update the $Q(S_t, A_t)$ using $Q(S_{t+1}, A_{t+1})$
- After that, $S_t \leftarrow S(t+1), A_t \leftarrow A(t+1)$.

Q-learning

- The 'q' stands for quality
 - how useful a given action is in gaining some future reward.
- Commonly used model free approach
 - Can be used for building self-playing PacMan agent
- Revolve around the notion of updating Q values
 - Which denotes the value of doing action a in state s

$$Q(s_t, a_t) \leftarrow (1 - \alpha) \cdot \underbrace{Q(s_t, a_t)}_{\text{old value}} + \underbrace{\alpha}_{\text{learning rate}} \cdot \overbrace{\left(\underbrace{r_t}_{\text{reward}} + \underbrace{\gamma}_{\text{discount factor}} \cdot \underbrace{\max_a Q(s_{t+1}, a)}_{\text{estimate of optimal future value}} \right)}^{\text{learned value}}$$

- Learning rate
 - factor of 0 -> the agent learn nothing (exclusively exploiting prior knowledge)
 - Factor of 1->agent consider only the most recent information (ignoring prior knowledge to explore possibilities)
- Discount factor
 - factor of 0 ->the agent "myopic" (or short-sighted) by only considering current rewards
 - Factor of approaching 1 ->it strive for a long-term high reward
 - If meets or exceeds 1, the action values may diverge

Q-learning

Q-learning (off-policy TD control) for estimating $\pi \approx \pi_*$

Algorithm parameters: step size $\alpha \in (0, 1]$, small $\varepsilon > 0$

Initialize $Q(s, a)$, for all $s \in \mathcal{S}^+, a \in \mathcal{A}(s)$, arbitrarily except that $Q(\text{terminal}, \cdot) = 0$

Loop for each episode:

 Initialize S

 Loop for each step of episode:

 Choose A from S using policy derived from Q (e.g., ε -greedy)

 Take action A , observe R, S'

$Q(S, A) \leftarrow Q(S, A) + \alpha [R + \gamma \max_a Q(S', a) - Q(S, A)]$

$S \leftarrow S'$

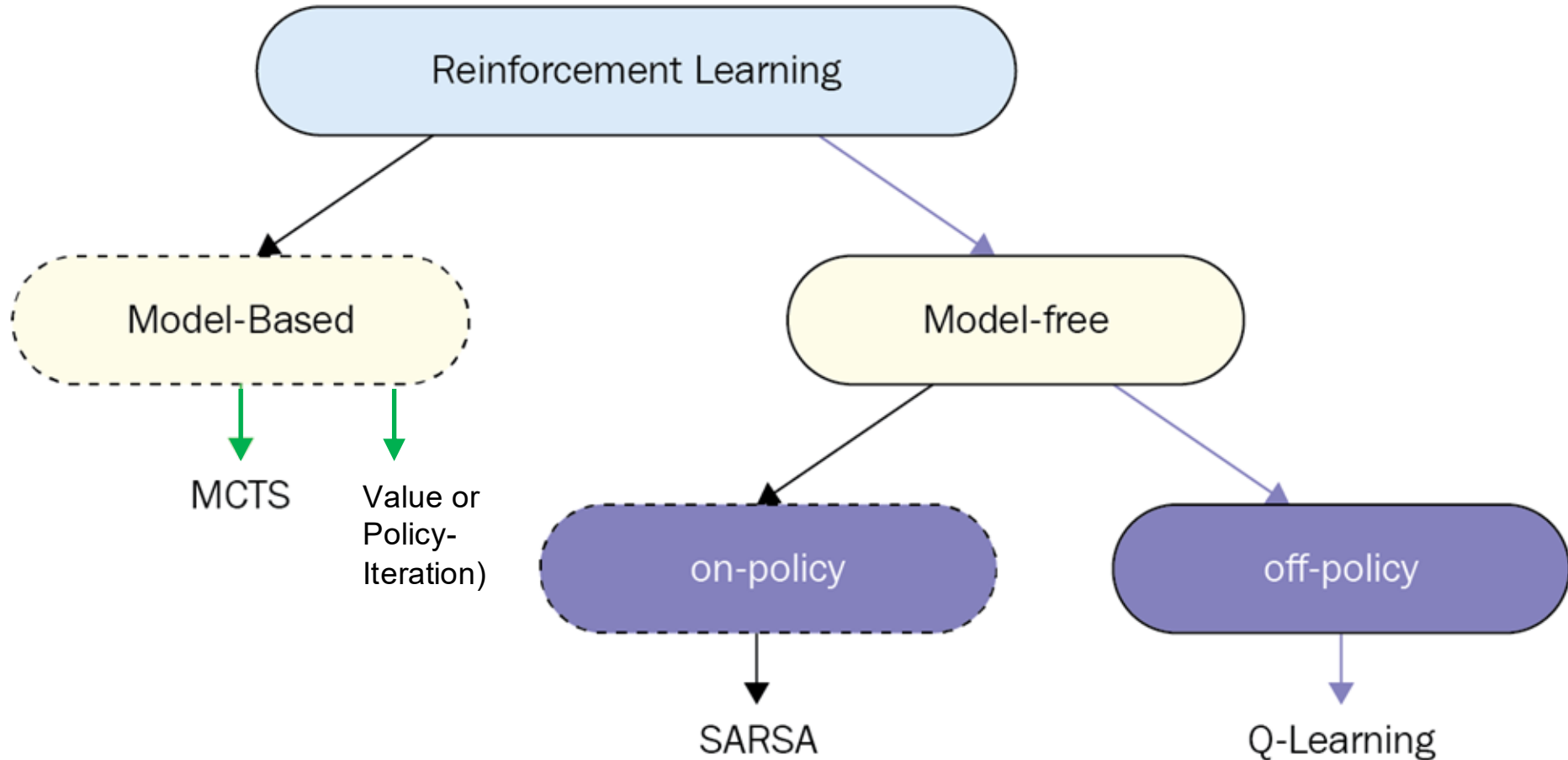
 until S is terminal

- will always choose the current A_t in current state S_t .
- estimate the biggest $Q(S_{t+1}, a)$ of next state.
- But when we go the next loop, we will choose the action a again.
- Whether a is A_{t+1} chosen before, it is not decided.

SARSA vs Q-Learning

- In Q-learning,
 - the agent starts out in state **S**,
 - performs action **A**,
 - sees what the highest possible reward is for taking any action from its new state, **T**,
 - and updates its value for the state **S**-action **A** pair based on this new highest possible value.
- In SARSA,
 - the agent starts in state **S**,
 - takes action **A** and gets a reward,
 - then moves to state **T**,
 - takes action **B** and gets a reward,
 - and then goes back to update the value for **S-A**
 - based on the actual value of the reward it received from taking action **B**.

SARSA vs Q-learning



Note: Monte Carlo tree search (MCTS) is a type of model-based RL

Recall

Value iteration

$$V_{i+1}(s) \leftarrow \max_a \sum_{s'} T(s, a, s') [R(s, a, s') + \gamma V_i(s')]$$

Q-Value iteration

$$Q_{i+1}(s, a) \leftarrow \sum_{s'} T(s, a, s') [R(s, a, s') + \gamma \max_{a'} Q_i(s', a')]$$

Policy iteration

$$V_{i+1}^{\pi_k}(s) \leftarrow \sum_{s'} T(s, \pi_k(s), s') [R(s, \pi_k(s), s') + \gamma V_i^{\pi_k}(s')]$$

$$\pi_{k+1}(s) = \arg \max_a \sum_{s'} T(s, a, s') [R(s, a, s') + \gamma V^{\pi_k}(s')]$$

3	0.812	0.868	0.912	
2	0.762		0.660	
1	0.705	0.655	0.611	0.388
	1	2	3	4

3	→	→	→	
2	↑		↑	
1	↑	←	←	←
	1	2	3	4

Model-based & Known Environment

TDL in RL

Temporal Difference Learning SARSA (on-policy)

Demo

$$Q(s_t, a_t) = Q(s_t, a_t) + \alpha(r_{t+1} + \gamma Q(s_{t+1}, a_{t+1}) - Q(s_t, a_t))$$

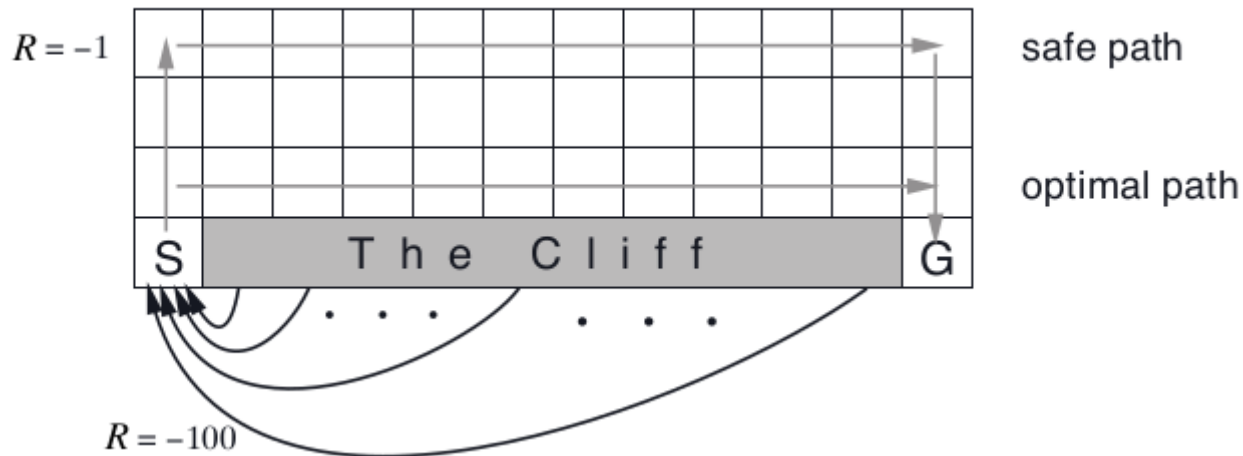
Q-Learning (off-policy)

$$Q(s_t, a_t) \leftarrow (1 - \alpha) \cdot \underbrace{Q(s_t, a_t)}_{\text{old value}} + \underbrace{\alpha}_{\text{learning rate}} \cdot \overbrace{\left(\underbrace{r_t}_{\text{reward}} + \underbrace{\gamma}_{\text{discount factor}} \cdot \underbrace{\max_a Q(s_{t+1}, a)}_{\text{estimate of optimal future value}} \right)}^{\text{learned value}}$$

Model-free &
no info about environment (explore or exploit)

Cliff-walking example

- The grid-world is of size 4x12, where the cliff is on the last row of size 10 between the start and end state. Each step that takes the agent to fall inside cliff will receive -100 reward, otherwise -1.
- We will take $\epsilon=0.1$ in our experiments. At each step, the agent has 10% of not following the greedy action (or the action with the highest Q value).



Cliff-walking example

- **SARSA: will choose safe path**
 - it learns that the agent might fall into the cliff
 - would lead to a large negative reward,
 - so it lowers the Q-values of those state-action pairs accordingly.
- **Q-learning: will choose optimal path**
 - learns values for the optimal policy,
 - that which travels right along the edge of the cliff.
 - occasionally falling off the cliff because of the “epsilon-greedy” action selection

Mouse vs Cliff example

Let's look at a simple scenario, a mouse is trying to get to a piece of cheese. Additionally, there is a cliff in the map that must be avoided, or the mouse falls, gets a negative reward, and has to start back at the beginning.

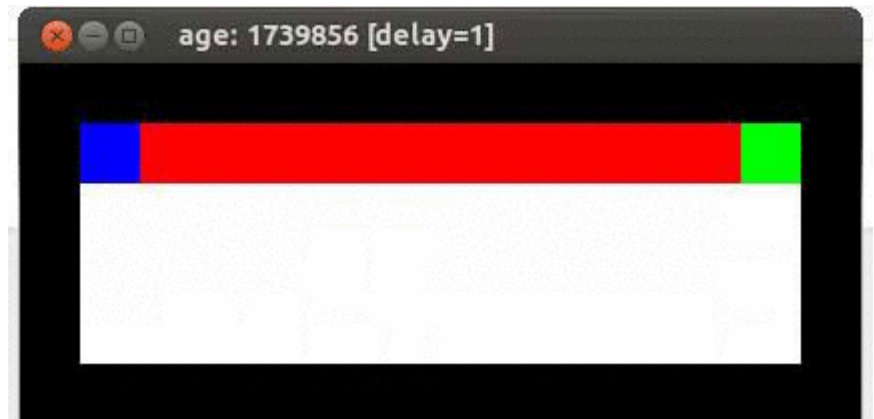
Q-learning:

- assumes that the mouse is following the optimal control strategy
- The solution that the mouse ends up with is running along the edge of the cliff and occasionally jumping off and plummeting to its death.



SARSA:

- the agent's *actual* control strategy is taken into account when learning
- best path is not to run along the edge of the cliff straight to the cheese but to get far away from the cliff and then work its way over safely



Next Lecture: Machine Learning

